

A I - P O W E R E D

Character Name Generator

System Architecture & Implementation Guide

A production-grade modular AI content generation system built
with LLM prompt engineering, Python, and Streamlit

Document Version 2.0

Complete Technical Reference for Development, Deployment & Evaluation

1. Project Overview

1.1 Objective

The **AI Character Name Generator** is a full-stack AI-powered application designed to assist writers, game designers, and creative professionals in generating **contextually rich, culturally informed, and stylistically aligned** character names for novels, scripts, games, and other narrative media.

The system intelligently combines user-defined parameters — such as genre, personality traits, cultural inspiration, and narrative tone — with a carefully engineered prompt pipeline to produce names that are not just random strings, but **semantically meaningful identities** for fictional characters.

1.2 Core Value Proposition

Pain Point Solved	How This System Addresses It
Hours lost brainstorming names	Instant generation of 5–10 semantically relevant names per query
Culturally inaccurate names	Cultural filters ensure names match regional/historical contexts
Generic, repetitive results	Post-processing deduplication and relevance scoring
No personality alignment	Trait-based prompting ensures names reflect character psychology
No backstory context	Each name ships with a meaning and a short character sketch

1.3 Design Philosophy

Core Principle: Leverage pre-trained LLM intelligence + structured prompt engineering + a lightweight post-processing logic layer. No custom model training required.

This approach is chosen deliberately over fine-tuning or training a custom model for three key reasons:

- Zero training time — the system is deployable from day one.
- High output quality — state-of-the-art LLMs already possess deep linguistic and cultural knowledge.
- Scalable architecture — swapping the LLM backend (local → cloud) requires changing only one configuration flag.
- Real-world alignment — this mirrors how production AI products are built in industry.

2. System Architecture

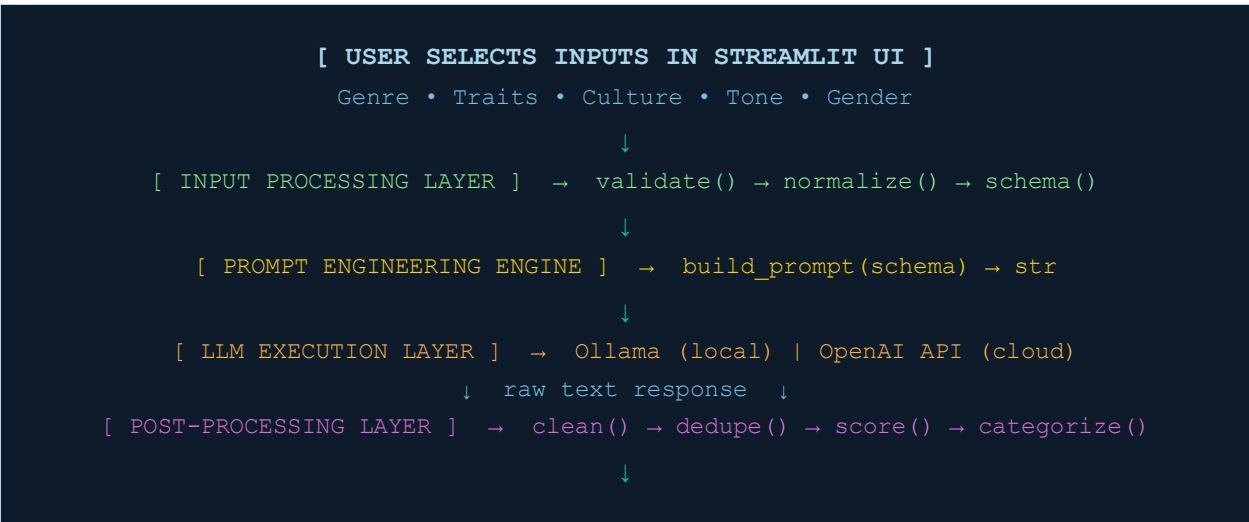
2.1 High-Level Architecture Overview

The system is structured as a **six-layer pipeline**, each layer having a single, clearly defined responsibility. This separation of concerns makes the codebase maintainable, testable, and extensible.

#	Layer	Responsibility	Tech Component
1	User Interface	Renders input controls, displays results, manages session state	Streamlit
2	Input Processing	Validates, normalizes, and structures raw user inputs into a clean schema	utils.py
3	Prompt Engine	Transforms the structured schema into a precise LLM instruction string	prompt_engine.py
4	LLM Execution	Routes the prompt to the active LLM backend; returns raw completion text	llm_handler.py
5	Post-Processing	Cleans, deduplicates, scores, and categorizes the raw LLM output	post_processing.py
6	Output Rendering	Displays structured name cards with export/save capabilities in the UI	Streamlit + app.py

2.2 Data Flow Diagram

The following describes the complete data journey for a single user request:



[OUTPUT RENDERED AS CARDS IN STREAMLIT] → Name | Meaning | Description | Category

2.3 Technology Stack

Category	Technology	Purpose & Justification
Frontend + Backend	Streamlit 1.x	Rapid Python-native UI; no JavaScript required; perfect for AI demos and academic projects
Language	Python 3.10+	Industry standard for AI/ML; rich ecosystem of NLP and LLM libraries
LLM Backend (Local)	Ollama + Mistral 7B	Zero API cost; offline capable; deterministic for demos; lightweight (4GB VRAM)
LLM Backend (Cloud)	OpenAI GPT-4o-mini	Fallback for high-quality output; used when Ollama is unavailable
Package Management	pip + requirements.txt	Standard Python dependency management; easily reproducible environments
Version Control	Git + GitHub	Source control, collaboration, and project submission

3. Functional Modules — In-Depth

3.1 User Input Module (app.py)

This module is the entry point of the application. It renders all input controls using Streamlit widgets and is responsible for collecting a well-formed set of user preferences before triggering the generation pipeline.

3.1.1 Input Parameters

Parameter	Widget Type	Example Values	Required?	Default
Genre	st.selectbox	Fantasy, Sci-Fi, Horror, Romance, Historical	Yes	Fantasy
Traits	st.text_input	Brave, cunning, melancholic, ruthless	Yes	—
Cultural Influence	st.selectbox	Indian, Norse, Japanese, Celtic, Arabic	Yes	Western

Gender	st.radio	Male, Female, Non-binary, Any	No	Any
Tone	st.selectbox	Dark, Whimsical, Epic, Subtle, Mythical	Yes	Epic
Number of Names	st.slider	1 – 15	No	10
Style Preset	st.selectbox	Custom, GoT-style, Anime, Mythological	No	Custom

3.1.2 Implementation Snippet

```
# app.py – Input Module
genre    = st.selectbox('Genre', ['Fantasy', 'Sci-Fi', 'Horror', 'Romance', 'Historical Fiction'])
traits   = st.text_input('Character Traits (comma-separated)', placeholder='e.g., brave, cunning')
culture  = st.selectbox('Cultural Influence', ['Indian', 'Norse', 'Japanese', 'Celtic', 'Arabic', 'Western'])
gender   = st.radio('Gender (optional)', ['Any', 'Male', 'Female', 'Non-binary'])
tone     = st.selectbox('Narrative Tone', ['Epic', 'Dark', 'Whimsical', 'Subtle', 'Mythical'])
n_names  = st.slider('Number of Names', min_value=3, max_value=15, value=10)

if st.button('Generate Names'):
    schema = process_inputs(genre, traits, culture, gender, tone, n_names)
    prompt = build_prompt(schema)
    raw    = call_llm(prompt)
    names  = post_process(raw)
    render_output(names)
```

3.2 Input Processing Layer (utils.py)

The input processing layer sits between the UI and the prompt engine. It acts as a **data sanitization and normalization gateway** — ensuring that no malformed, empty, or inconsistent inputs ever reach the LLM.

3.2.1 Responsibilities

- Validate that required fields (genre, traits, tone) are non-empty.
- Normalize strings: strip whitespace, convert to lowercase, remove duplicate traits.
- Expand abbreviations and common shorthand (e.g., 'sci-fi' → 'science fiction').
- Combine style preset overrides with manual inputs where applicable.
- Return a clean, typed Python dictionary (the 'schema') ready for the prompt engine.

3.2.2 Output Schema

```
# Example normalized schema output from utils.py
{
  "genre": "dark fantasy",
  "traits": ["ruthless", "intelligent", "haunted by past"],
  "culture": "indian",
  "gender": "male",
  "tone": "mysterious",
  "count": 10
}
```

3.3 Prompt Engineering Engine (prompt_engine.py)

Why This Is The Core System: The quality of an LLM output is almost entirely determined by the quality of the prompt. This module is the intellectual heart of the project — it is where AI product design meets linguistics and software engineering.

The prompt engine dynamically constructs a **structured, constraint-rich natural language instruction** from the normalized schema. It uses several advanced prompt engineering techniques:

3.3.1 Prompt Engineering Techniques Used

- **Role Prompting:** Instructing the LLM to act as an expert fantasy writer/linguist.
- **Constraint Injection:** Embedding hard constraints (culture, tone, genre) as explicit requirements.
- **Output Format Specification:** Defining exact fields (Name, Meaning, Description) to enforce parseable output.
- **Context Stacking:** Layering multiple context signals to achieve semantic alignment.
- **Persona-Trait Linking:** Ensuring trait keywords semantically influence phonetic and etymological name choices.

3.3.2 Master Prompt Template

```
You are an expert author and cultural linguist specializing in character naming.
Your task is to generate {count} unique, original character names for a {genre} novel.

REQUIREMENTS:
- Cultural Influence : {culture}
- Character Traits   : {traits}
- Narrative Tone     : {tone}
- Gender            : {gender}
```

```
For each name, provide EXACTLY this structure:
Name      : <name>
Meaning   : <etymological meaning or invented meaning>
Description : <2-sentence character sketch that aligns with the traits>
Category  : <Hero | Villain | Neutral>

CONSTRAINTS:
- Names must sound authentic to the specified culture.
- No two names should share the same root or sound.
- Names must be pronounceable and memorable.
- Do not include any real-world famous names.
- Output ONLY the structured list. No preamble or explanation.
```

3.4 LLM Execution Layer (llm_handler.py)

This layer abstracts the underlying LLM backend behind a single function call: `call_llm(prompt: str) → str`. The calling code never needs to know whether the LLM is running locally or in the cloud.

3.4.1 Dual-Mode Architecture

Mode	Configuration	Model Used	Best For
LOCAL	USE_LOCAL_MODEL = True	Ollama / Mistral 7B	Demo reliability, offline use, zero cost
CLOUD	USE_LOCAL_MODEL = False	OpenAI GPT-4o-mini	Higher output quality, production deployments

3.4.2 Implementation

```
# llm_handler.py
USE_LOCAL_MODEL = True  # Toggle between local and cloud

def call_llm(prompt: str) -> str:
    if USE_LOCAL_MODEL:
        return _call_ollama(prompt)
    else:
        return _call_openai(prompt)

def _call_ollama(prompt: str) -> str:
    import ollama
    response = ollama.chat(
```

```
        model='mistral',
        messages=[{'role': 'user', 'content': prompt}]
    )
    return response['message']['content']

def _call_openai(prompt: str) -> str:
    from openai import OpenAI
    client = OpenAI() # reads OPENAI_API_KEY from env
    res = client.chat.completions.create(
        model='gpt-4o-mini',
        messages=[{'role': 'user', 'content': prompt}],
        temperature=0.85,
        max_tokens=2048
    )
    return res.choices[0].message.content
```

3.5 Post-Processing Layer (post_processing.py)

Raw LLM output is inherently noisy and inconsistently formatted. The post-processing layer applies four sequential operations to transform raw text into clean, structured data objects ready for UI rendering.

3.5.1 Processing Pipeline

- Stage 1 — Parsing: Split raw output on delimiter patterns; extract Name, Meaning, Description, Category fields using regex.
- Stage 2 — Cleaning: Strip leading/trailing whitespace; remove markdown artifacts; fix encoding issues; validate all four fields are present.
- Stage 3 — Deduplication: Compute Levenshtein distance between all name pairs; discard names with similarity > 80%.
- Stage 4 — Scoring: Assign a quality score (0–100) based on three sub-scores:
 - Relevance Score: Does the name phonetically align with the culture?
 - Uniqueness Score: How distinct is this name from all others in the batch?
 - Readability Score: How easily can an average reader pronounce and remember it?
- Stage 5 — Categorization: Assign Hero / Villain / Neutral based on LLM-provided category or fallback keyword matching.

3.5.2 Output Data Structure

```
# Each processed name is represented as:
@dataclass
class CharacterName:
    name: str
```

```

meaning:      str
description:  str
category:     str    # 'Hero' | 'Villain' | 'Neutral'
score:       float  # 0.0 - 100.0

```

3.6 Output Rendering Module (app.py — render section)

The rendered output displays each character name in a structured card layout within Streamlit. Cards are colour-coded by category and sorted by quality score descending.

```

# render_output(names: list[CharacterName])
for name_obj in sorted(names, key=lambda x: x.score, reverse=True):
    color =
    {'Hero': '#1ABC9C', 'Villain': '#E74C3C', 'Neutral': '#3498DB'}[name_obj.category]
    st.markdown(f'''
        <div style="border-left: 5px solid {color}; padding: 10px; margin: 8px 0;">
            <h3>{name_obj.name} — {name_obj.category}</h3>
            <p><b>Meaning:</b> {name_obj.meaning}</p>
            <p>{name_obj.description}</p>
            <small>Quality Score: {name_obj.score:.0f}/100</small>
        </div>''', unsafe_allow_html=True)

```

4. Project Structure & File Reference

4.1 Recommended Folder Layout

```

character-name-generator/
|
├─ app.py                # Main Streamlit entry point; UI layer + orchestration
├─ prompt_engine.py      # Prompt template builder; core engineering logic
├─ llm_handler.py        # LLM abstraction layer; Ollama + OpenAI backends
├─ post_processing.py     # Output cleaning, deduplication, scoring, categorization
├─ utils.py              # Input validation, normalization, schema builder
|
├─ config.py             # Global settings: model choice, API keys, defaults
├─ requirements.txt       # Python dependencies
├─ .env                  # API keys (never commit to Git)
├─ .gitignore            # Excludes .env, __pycache__, .streamlit cache
|

```

```
├─ tests/                                # Unit tests for each module
│   ├── test_utils.py
│   ├── test_prompt_engine.py
│   └── test_post_processing.py
│
├─ docs/                                # This document + architecture diagrams
│   └── architecture.md
```

4.2 File Responsibilities Summary

File	Detailed Responsibility
app.py	Entry point. Initializes Streamlit page config, renders sidebar controls, calls each pipeline stage in sequence, renders output cards, handles session state for saved names.
prompt_engine.py	Contains the master prompt template string and the build_prompt(schema) function. Handles style preset overrides, gender-neutral language adjustments, and output format specification.
llm_handler.py	Provides a single call_llm(prompt) interface. Internally routes to _call_ollama() or _call_openai() based on USE_LOCAL_MODEL flag. Handles timeouts and API errors gracefully.
post_processing.py	Implements the five-stage output pipeline: parse → clean → deduplicate → score → categorize. Returns a sorted list of CharacterName dataclass objects.
utils.py	Input validation guards, string normalization utilities, trait tokenizer, genre/culture expander, and the process_inputs() schema builder function.
config.py	Central configuration: USE_LOCAL_MODEL flag, default model names, max token limits, UI theme colors, export file paths.

5. Setup, Installation & Deployment

5.1 Prerequisites

- Python 3.10 or higher installed and added to PATH.
- pip package manager (bundled with Python 3.10+).
- Git installed for version control.
- For LOCAL mode: Ollama installed (ollama.com) and Mistral model pulled.
- For CLOUD mode: OpenAI API key set as the OPENAI_API_KEY environment variable.

5.2 Local Deployment (Recommended for Demos)

Step 1 — Clone the Repository

```
git clone https://github.com/your-username/character-name-generator.git
cd character-name-generator
```

Step 2 — Create & Activate Virtual Environment

```
python -m venv venv
# On Windows:   venv\Scripts\activate
# On Mac/Linux: source venv/bin/activate
```

Step 3 — Install Python Dependencies

```
pip install -r requirements.txt
```

Step 4 — Install & Configure Ollama (Local Mode)

```
# Download Ollama from: https://ollama.com
# Then pull the Mistral model (~4GB):
ollama pull mistral
ollama serve # Start the local inference server
```

Step 5 — Run the Application

```
streamlit run app.py
# App opens automatically at: http://localhost:8501
```

5.3 requirements.txt

```
streamlit>=1.28.0
ollama>=0.1.6
openai>=1.0.0
python-dotenv>=1.0.0
Levenshtein>=0.23.0
dataclasses-json>=0.6.0
```

5.4 Cloud Deployment (Optional)

For cloud deployment on **Streamlit Community Cloud**:

- Push the project to a public GitHub repository.
- Visit share.streamlit.io and connect the repository.
- Add OPENAI_API_KEY in the Streamlit Secrets Manager.
- Set USE_LOCAL_MODEL = False in config.py (Ollama is unavailable on cloud).
- Deploy — Streamlit handles hosting, scaling, and HTTPS automatically.

6. Advanced Features & Innovation Layer

6.1 Style Presets System

Style presets allow users to apply pre-configured prompt overlays that mimic the naming conventions of popular fictional universes or cultural traditions. When a preset is selected, it **overrides the individual tone and cultural inputs** with a curated combination.

Preset Name	Prompt Overlay Applied	Example Output Name
Game of Thrones Style	Medieval European, harsh phonetics, house-based surnames	Vaelorn Duskmantle
Anime Style	Japanese phonetics, nature/virtue meanings, two-syllable first names	Kazuho Ryuusei
Mythological Indian	Sanskrit roots, deity connotations, Vedic/Puranic etymologies	Vikramaditya Suryavansh
Norse Mythology	Proto-Germanic roots, runic symbolism, compound kenning structure	Bjornulfr Ironsong
Custom	Uses all individual field inputs as-is	(user-defined)

6.2 Hybrid Name Construction Engine

An optional algorithmic sub-system that generates names through **morpheme-level composition** rather than relying solely on the LLM. This creates a richer diversity of names and is faster for bulk generation.

```
# Morpheme tables for 'Dark Fantasy Indian' combination:
PREFIXES = ['Kir', 'Var', 'Zor', 'Ash', 'Khal', 'Drav']
ROOTS    = ['ath', 'mor', 'ven', 'sul', 'ran', 'drak']
SUFFIXES = ['ius', 'ara', 'on', 'esh', 'ai', 'orn']
```

```
def construct_name(prefix, root, suffix):
    return f'{prefix}{root}{suffix}' # e.g., 'Zorathon', 'Khaldrakesh'
```

The hybrid engine is invoked as a fallback if the LLM returns fewer names than requested, or as an enhancement to post-processed results.

6.3 Save & Export Feature

Users can save favourite generated names to a session-persistent list, and export the full list to standard formats:

- Export as TXT — one name per line with meaning and description.
- Export as CSV — structured columns for spreadsheet use.
- Export as JSON — machine-readable format for integration with game engines or writing apps.

```
import pandas as pd

def export_csv(names: list[CharacterName]) -> bytes:
    df = pd.DataFrame([vars(n) for n in names])
    return df.to_csv(index=False).encode('utf-8')

# In app.py:
st.download_button('Download CSV', export_csv(saved_names), 'names.csv', 'text/csv')
```

7. Evaluation Framework & Viva Preparation

7.1 How to Evaluate Output Quality

Since there is no single 'correct' answer for creative name generation, quality is assessed across three measurable dimensions:

Dimension	What It Measures	Weight in Score
Contextual Relevance	Does the name fit the genre, tone, and cultural context?	40%

Uniqueness	Is the name distinct from others in the batch? From real-world names?	35%
Readability	Is the name pronounceable and memorable for an average reader?	25%

7.2 Key Viva Questions & Model Answers

Q: Why didn't you train your own model?

A: Training a custom model requires large labeled datasets, significant compute resources (GPUs), weeks of training time, and deep ML expertise — none of which add value to this specific problem. Pre-trained LLMs already have extensive knowledge of linguistics, culture, and creative writing. Our contribution is the product design: the prompt engineering system, the post-processing pipeline, and the modular architecture. This mirrors real-world AI product development.

Q: What is prompt engineering and why does it matter?

A: Prompt engineering is the discipline of crafting natural language instructions that guide an LLM to produce outputs that meet specific quality, format, and style requirements. It matters because the same LLM will produce dramatically different results depending on how the prompt is structured. Our prompt engine uses role assignment, constraint injection, and output format specification to reliably produce structured, high-quality results.

Q: How does the system handle bad LLM output?

A: The post-processing layer is designed specifically for this. It uses regex-based parsing to extract structured fields even from imperfectly formatted text, filters out malformed entries, deduplicates similar names, and scores each result — discarding those below a quality threshold. The system degrades gracefully: if only 6 of 10 requested names pass quality checks, 6 are displayed rather than showing the user malformed data.

8. Future Roadmap

Phase	Feature	Description	Priority
1	User Accounts	Allow users to sign up, save name collections, and build character libraries across sessions.	High
1	Name History	Persistent database (SQLite) to store all generated names with timestamps and input params.	High

2	World-Building Assistant	Extend the system to generate place names, faction names, and lore fragments alongside character names.	Medium
2	Relationship Graph	Allow users to link characters and visualize relationships (ally, rival, family) as an interactive graph.	Medium
3	Fine-Tuned Model	Optional: fine-tune a small model (Mistral 7B) on curated fantasy name datasets for niche genres.	Low
3	API Endpoint	Expose the generation pipeline as a REST API (FastAPI) for integration with game engines and writing tools.	Low

9. Project Deliverables Checklist

Deliverable	Status Indicator
Working Streamlit application (local deployment)	☐ Not Started ☐ In Progress ☐ Done
Modular codebase with all 5 Python modules	☐ Not Started ☐ In Progress ☐ Done
Prompt Engineering Engine (prompt_engine.py)	☐ Not Started ☐ In Progress ☐ Done
Post-Processing Pipeline with scoring	☐ Not Started ☐ In Progress ☐ Done
Dual-mode LLM handler (local + cloud fallback)	☐ Not Started ☐ In Progress ☐ Done
Export feature (CSV / TXT / JSON)	☐ Not Started ☐ In Progress ☐ Done
Unit tests for core modules	☐ Not Started ☐ In Progress ☐ Done
Architecture diagram (visual)	☐ Not Started ☐ In Progress ☐ Done
Presentation slides (PPT) with demo screenshots	☐ Not Started ☐ In Progress ☐ Done
This comprehensive project document	☐ Not Started ☐ In Progress ☒ Done